

stimpack: a modular framework for precise multisensory stimulus generation in systems neuroscience

Minseung Choi¹✉, Joshua B. Melander¹, Steven G. Herbst², Carl F. R. Wienecke^{1,3}, Heberto Mayorquin⁴, Alex Y. Hao¹, Manze Zhang¹, Jacob C. Simon¹, David Tadres¹, David D. Au¹, Bella E. Brezovec¹, Stephen A. Baccus¹, Thomas R. Clandinin^{1,5*}, and Maxwell H. Turner^{1,6*✉}

¹ Department of Neurobiology, Stanford University, USA ² Department of Electrical Engineering, Stanford University, USA ³ Department of Neurobiology, Harvard Medical School, USA ⁴ CatalystNeuro, USA ⁵ Biohub, San Francisco, USA ⁶ Department of Biological Sciences, University at Albany, SUNY, USA ✉ Corresponding author * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ✉
- [Repository](#) ✉
- [Archive](#) ✉

Editor: [Open Journals](#) ✉

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

A central goal of neuroscience is to understand how animals engage with the natural world. Studying this experimentally requires the ability to present stimuli that capture the complexity of dynamic sensory inputs during animal behavior. But performing such experiments across labs, and interpreting the resulting data, can be challenging because doing so requires highly flexible and precisely defined stimuli that can be synchronized with other measurement tools, and can be reconstructed to facilitate data analysis.

To address these challenges, we introduce *stimpack*, a Python framework for open- and closed-loop stimulus delivery. *stimpack* is a hardware-agnostic stimulus rendering environment where a single protocol simultaneously drives perspective-corrected displays, sound playback, analog outputs, and closed-loop position updates. Data are written through pluggable backends, including the neuroscience community standard Neurodata Without Borders (NWB) (Rübel et al., 2022; Teeters et al., 2015). Everything specific to a laboratory or experimental setup (e.g. screen geometry, devices, and stimuli) is coded in a lab- or user-specific *labpack*. One protocol library runs across rigs with different screen geometries and hardware, allowing experiments to be replicated precisely across laboratories.

Statement of need

stimpack was designed for laboratories that position visual displays around a stationary animal and couple the outputs of these displays to animal behavior in closed loop. In this context, *stimpack* satisfies three important requirements. First, with *stimpack* the same visual stimulus can be shown across different presentation geometries, including setups with multiple displays or curved screens. A stimulus can be specified in terms of visual angles relative to the subject, as in parametric psychophysics, or specified by Euclidean positions and sizes, as in a virtual environment. In either case, each display renders the stimulus at perspective-corrected angles (Figure 2). Second, the same package can coordinate multisensory stimulus delivery, closed-loop control, and analog device control, allowing these devices to be synchronized. Finally, *stimpack*'s files are valid NWB, so acquired data can be added with *pynwb* or *NeuroConv* (Mayorquin et al., 2025), browsed with *NeuroSift* (Magland et al., 2024), and archived on

41 DANDI.

42 State of the field

43 We created `stimpack` to fill a gap by enabling flexible display geometries, native closed-loop
44 control, modular device handling, and simple yet powerful stimulus design. Existing software
45 packages for experiment control and visual stimulus delivery typically satisfy some, but not all,
46 of these requirements.

47 Some visual display packages were built for **precise parametric control of a stimulus shown to a**
48 **participant**. These packages make it easy to design visual stimuli, but can be difficult to extend
49 to new screen geometries or incorporate closed-loop control based on behavioral feedback. For
50 example, Psychtoolbox (Brainard, 1997; Pelli, 1997) and PsychoPy (Peirce et al., 2019) are
51 designed around a display viewed roughly head-on, not around several surfaces at arbitrary
52 orientations, and cannot readily be coupled to movement. A similar static-observer display
53 model underlies tools for head-fixed physiology such as MonkeyLogic, PLDAPS, MWorks, and
54 Rigbox (Bhagat et al., 2020; Eastman & Huk, 2012; Hwang et al., 2019; The MWorks Project,
55 2026), as well as in the Python packages Vision Egg (Straw, 2008) and QDSpy (Franke et al.,
56 2019).

57 Other software packages have been purpose-built for use cases that involve **closed-loop control**
58 **in a virtual environment**. These packages, while often delivering good performance, are often
59 designed with specific hardware setups or requirements. For example, Stytra (Štih et al.,
60 2019) and vxPy (Hladnik, 2026) have been used for a single preparation (larval zebrafish)
61 while FreemoVR (Stowers et al., 2017) extended similar functionality to freely moving animals.
62 Modular, custom LED arenas designed for work in fruit flies (Isaacson et al., 2022; Reiser
63 & Dickinson, 2008) reach kilohertz binary frame rates with specific hardware requirements
64 and a fixed geometry and resolution. Each of these packages allows for closed-loop control of
65 parameterized stimuli, but does not directly support transferring experimental specifications to
66 other rig geometries.

67 The existing packages that are most similar to `stimpack` can be extended to **diverse display**
68 **types and geometries**. For example, ViRMEn (Aronov & Tank, 2014) and BonVision (Lopes
69 et al., 2021) allow for curved displays, including conical and spherical screens. Bonsai (Lopes
70 et al., 2015) handles closed-loop control and mixed devices, but trial structure and logging are
71 left to the workflow's author. Finally, game engines like Unity or Unreal that have been used
72 in neuroscience supply worlds rather than calibrated parametric stimuli, whether used directly
73 (Jangraw et al., 2014) or wrapped for dome projection (Shapcott et al., 2025).

74 `stimpack` satisfies all these requirements, with a focus on a flexible architecture. Everything
75 rig-specific is configured in a laboratory-specific labpack rather than inside experiment code, so
76 **one protocol can be used across rigs and preparations**. To our knowledge, no existing package
77 combines observer-centered perspective correction across an arbitrary number of flat screens
78 and curved surfaces in one rig, closed-loop coupling to any locomotor tracker, non-visual
79 outputs (sound, analog control) addressed from the same protocol, and trial-parameterized
80 metadata saved in a community data standard by default.

81 Software design

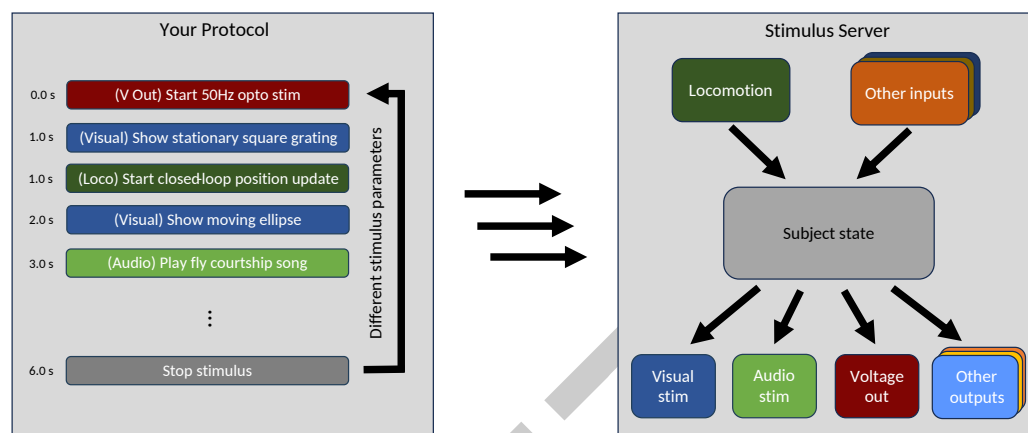


Figure 1: stimpack's architecture. A protocol names the module for each call. Inputs update a subject state that outputs follow, such that the closed loop does not pass through the client.

82 One module per capability

83 In stimpack, an experiment runs as a **client**, which executes the protocol and writes metadata
84 to the data file, and a **server**, which controls hardware devices, including display screens, as
85 a set of modules (Figure 1). This modular design choice allows a shared protocol library to
86 run on rigs whose hardware may differ. Four modules, defined by the capability rather than a
87 particular device, are included in stimpack and described below.

88 The visual module controls visual stimulus rendering and display. Stimulus shapes can be
89 painted with monochrome or RGB textures for stimuli like gratings, spatiotemporal white noise,
90 or movie frames. The visual module supports most display devices and perspective-corrected
91 rendering to any number of flat or curved screens. Temporal multiplexing packs up to three
92 stimulus timepoints into the color channels of each output frame, so a 120 Hz video signal
93 can drive a 360 Hz monochrome display, approaching the temporal regime of specialized LED
94 hardware (Reiser & Dickinson, 2008). During stimulus presentation, a small square in the
95 corner of the display inverts on every rendered frame to enable accurate synchronization with
96 other recorded data and to enable detection of dropped frames (Figure 2d).

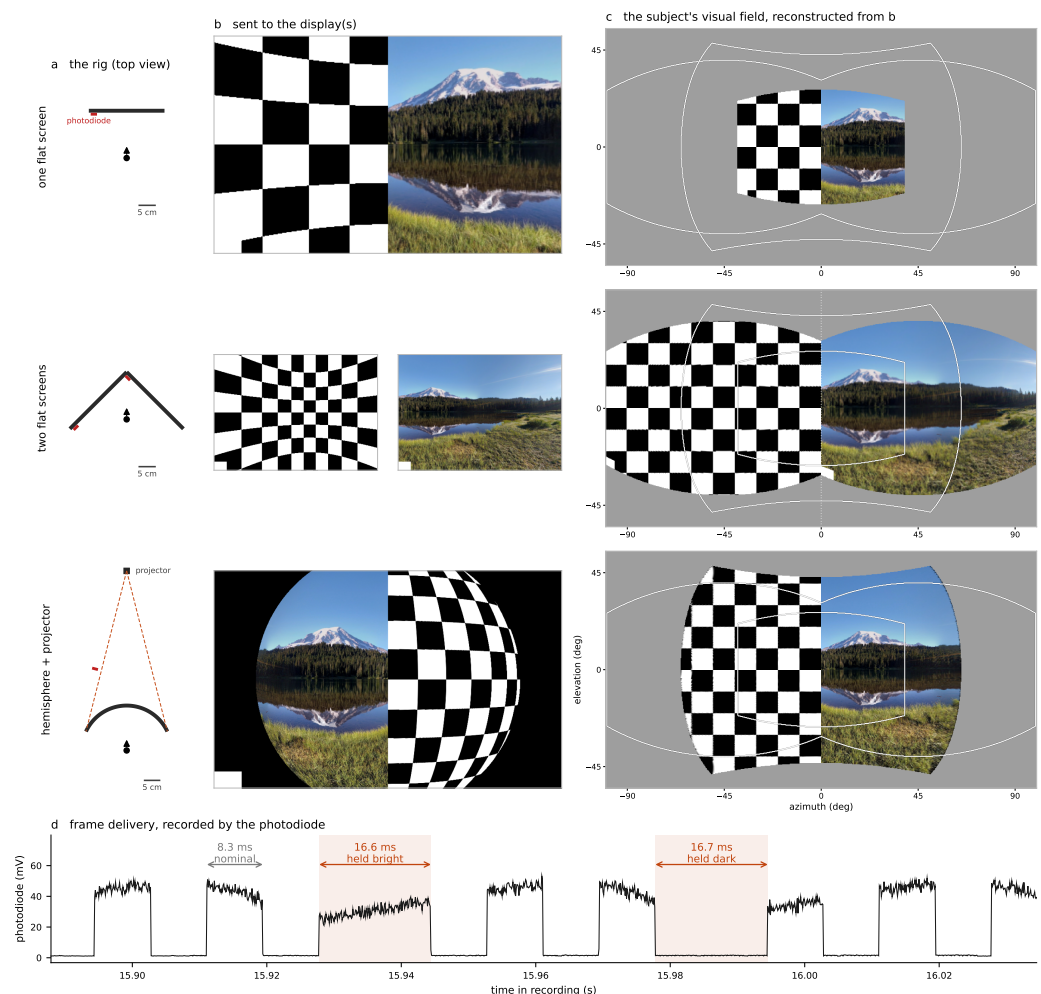


Figure 2: One stimulus specification, three display geometries, one visual experience. (a) Three rigs, to scale: one monitor; two monitors meeting ahead of the subject; a hemispherical screen lit by a projector. Red: the photodiode at each display's synchronization square. (b) The frames stimpack sends to the displays for one specified scene – a 10° checkerboard to the left and a 360° photograph to the right: stimpack warps the images according to the specification of each display. (c) The subject's visual field. Grey marks directions with no display surface. Faint outlines trace the coverages of the other two rigs for comparison across rows. (d) Frame delivery at the photodiode: nominal inversions every 8.3 ms (120 Hz), and two dropped frames, one held bright and one held dark.

The audio module plays sound stimuli through a sound card. The locomotion module handles locomotion-related behavior data for closed-loop feedback based on a lab-defined control function. The voltage_out module can send voltage waveforms to control devices (e.g. optogenetics, odor delivery, or reward).

Protocols, data, and the interface

A protocol is a Python class in the labpack that declares the parameters of a run and of the stimulus. Named presets can be saved and loaded to standardize protocol parameter sets. Users can also run an ensemble, which is a saved queue of protocols. Data are written through pluggable backends including NWB (Rübel et al., 2022; Teeters et al., 2015). stimpack includes a GUI that is a shell over these objects and identical on every rig (Figure 3).

the protocol: a Python class in the labpack (abridged)

```
class AudiovisualPairing(BaseProtocol):
    """A pulse song and a moving patch, started together."""

    def get_protocol_parameter_defaults(self):
        return {
            'pre_time': 0.5,
            'stim_time': 2.0,
            'tail_time': 1.0,
            'freq': [225.0, 120.0],
            'intensity': 0.0,
            'center': (0, 0),
            'speed': 60.0,
            'angle': 0.0,
            'volume': 0.5}

    def get_trial_parameters(self):
        ... # each trial's MovingPatch + PulseSong

# each trial, stimpack turns the descriptors into module calls:
server.target('visual').load_stim(name='MovingPatch', theta=..., ...)
server.target('audio').load_stim(name='PulseSong', freq=225.0, ...)
sleep(pre_time) # gray screen, silence
server.target('all').start_stim() # one start, both modalities
sleep(stim_time) # the patch sweeps, the song plays
server.target('all').stop_stim()
sleep(tail_time)
```

the GUI's Main tab, built from those declarations, mid-run

The GUI's Main tab displays the following parameters:

- Protocol:** AudiovisualPairing
- Param preset:** Default
- Run parameters:**
 - num_trials: 40
 - idle_color: 0.5
 - all_combinations: ☒
 - randomize_order: ☒
- Protocol parameters:**
 - pre_time: 0.5
 - stim_time: 2.0
 - tail_time: 1.0
 - freq: [225.0, 120.0] (highlighted with a red box and labeled 'This trial: freq: 225.0')
 - intensity: 0.0
 - center: (0, 0)
 - speed: 60.0
- Series #:** 1
- Subject:** subject-1
- Elapsed / Est:** 8 / 140s
- Trials run:** 2 / 40
- Buttons:** View, Record, Pause, Stop
- Status:** Recording series 1
- Note:**

Figure 3: From protocol code to a running experiment. Left: the AudiovisualPairing protocol, abridged, and the module calls that stimpack makes from its descriptors on each trial. Right: the GUI's Main tab mid-run, its parameter fields built from the class's own declarations. The list-valued freq (solid red box) sweeps across trials in randomized order.

107 Extensibility

108 stimpack is extensible at every layer. Stimulus classes load from the user-specific labpack at
109 runtime and are addressed exactly as built-ins; data backends, trackers, and DAQ drivers are
110 likewise labpack code. A new capability (e.g. a novel sensor or effector) is a subclass of the
111 same module base class the built-ins inherit.

112 Research impact statement

113 stimpack consolidates four earlier Clandinin lab packages: flystim (perspective-corrected
114 rendering), flyrpc (client-server messaging), visprotocol (protocols, metadata, GUI), and
115 multistim (auditory stimulation). None has been described in an archival publication, and
116 every author of those packages is an author here. The subscreen geometry is based on that
117 of flystim. The curved-screen path was checked against flymax, an earlier MATLAB-based
118 hemisphere-projector display in the lab, and its measured photometry. Predecessor versions
119 underlie published work on fly visual processing and behavior (Currier & Clandinin, 2025; Mano
120 et al., 2023; Turner et al., 2022), the most recent of which names flystim, visprotocol, and
121 stimpack itself. In addition, stimpack has been used to reconstruct spatiotemporal receptive
122 fields in mouse retina and visual cortex (Au et al., 2026; Weddington et al., 2026), including
123 the example in Figure 4. stimpack is in use in ongoing *Drosophila* experiments in the Clandinin
124 (Stanford), Turner (Albany), and Murthy (Princeton) laboratories, and in mouse work in the
125 Baccus and Mitra laboratories (Stanford).

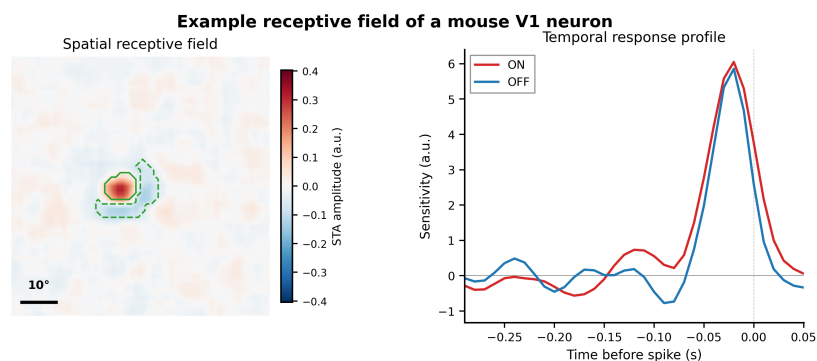


Figure 4: Spatiotemporal receptive field (STRF) of a mouse primary visual cortex neuron. The STRF of a cortical neuron recorded with a Neuropixels array was mapped with a white noise stimulus generated and rendered on two flat screens (similar to Figure 2, second row) with stimpack. The white noise stimulus had a 13° spatial correlation and a 33-ms temporal correlation. The eye was tracked to reconstruct the retinal image at a spatial scale of < 1° (Au et al., 2026).

AI usage disclosure

All primary architectural design decisions and implementations were made by authors. Generative AI (Claude Opus 4.8, Opus 5, and Fable 5 from Anthropic) was used for parts of code generation and editing (multisample anti-aliasing, curved screen, audio module, experiment GUI elements), code merging (curved screen, NWB data), documentation, testing scaffolding, and manuscript revision. Authors thoroughly reviewed, modified, and validated all AI-generated content.

Acknowledgements

We thank the Clandinin, Turner, Baccus, Mitra, and Murthy Labs for testing across rigs and preparations. This work was supported by Stanford Graduate Fellowships (MC, SGH), a DoD NDSEG Fellowship (MC), a Hertz Fellowship (SGH), a Swiss National Science Foundation Postdoc Mobility Fellowship (222161; DT), and NIH grants R00-EY032549, R01-EY022638, R01-EY022933, R01-EY025087, R01-EY034566 and P30-EY026877. TRC is a Biohub, San Francisco, Investigator.

References

- Aronov, D., & Tank, D. W. (2014). Engagement of neural circuits underlying 2D spatial navigation in a rodent virtual reality system. *Neuron*, 84(2), 442–456. <https://doi.org/10.1016/j.neuron.2014.08.042>
- Au, D. D., Melander, J. B., Weddington, J. C., Faragalla, Y., Alaoui, Z., Liu, S., Xu, Q., & Baccus, S. A. (2026). Single-camera, calibration-free gaze estimation using corneal reflections. *bioRxiv*. <https://doi.org/10.64898/2026.06.08.731021>
- Bhagat, J., Wells, M. J., Harris, K. D., Carandini, M., & Burgess, C. P. (2020). Rigbox: An open-source toolbox for probing neurons and behavior. *eNeuro*, 7(4), ENEURO.0406–19.2020. <https://doi.org/10.1523/ENEURO.0406-19.2020>
- Brainard, D. H. (1997). The Psychophysics Toolbox. *Spatial Vision*, 10(4), 433–436. <https://doi.org/10.1163/156856897X00357>
- Currier, T. A., & Clandinin, T. R. (2025). Infrequent strong connections constrain connectomic

- 153 predictions of neuronal function. *Cell*, 188(16), 4366–4381.e14. [https://doi.org/10.1016/j.](https://doi.org/10.1016/j.cell.2025.05.007)
154 [cell.2025.05.007](https://doi.org/10.1016/j.cell.2025.05.007)
- 155 Eastman, K. M., & Huk, A. C. (2012). PLDAPS: A hardware architecture and software toolbox
156 for neurophysiology requiring complex visual stimuli and online behavioral control. *Frontiers*
157 *in Neuroinformatics*, 6, 1. <https://doi.org/10.3389/fninf.2012.00001>
- 158 Franke, K., Maia Chagas, A., Zhao, Z., Zimmermann, M. J. Y., Bartel, P., Qiu, Y., Szatko,
159 K. P., Baden, T., & Euler, T. (2019). An arbitrary-spectrum spatial visual stimulator for
160 vision research. *eLife*, 8, e48779. <https://doi.org/10.7554/eLife.48779>
- 161 Hladnik, T. (2026). *vxPy: Multiprocess-based software for vision experiments in Python*.
162 <https://github.com/thladnik/vxPy>
- 163 Hwang, J., Mitz, A. R., & Murray, E. A. (2019). NIMH MonkeyLogic: Behavioral control
164 and data acquisition in MATLAB. *Journal of Neuroscience Methods*, 323, 13–21. <https://doi.org/10.1016/j.jneumeth.2019.05.002>
- 165
- 166 Isaacson, M., Ferguson, L., Loesche, F., Ganguly, I., Chen, J., Chiu, A., Liu, J., Dickson,
167 W., & Reiser, M. (2022). A high-speed, modular display system for diverse neuroscience
168 applications. *bioRxiv*. <https://doi.org/10.1101/2022.08.02.502550>
- 169 Jangraw, D. C., Johri, A., Gribetz, M., & Sajda, P. (2014). NEDE: An open-source scripting
170 suite for developing experiments in 3D virtual environments. *Journal of Neuroscience*
171 *Methods*, 235, 245–251. <https://doi.org/10.1016/j.jneumeth.2014.06.033>
- 172 Lopes, G., Bonacchi, N., Frazão, J., Neto, J. P., Atallah, B. V., Soares, S., Moreira, L., Matias,
173 S., Itskov, P. M., Correia, P. A., Medina, R. E., Calcaterra, L., Dreosti, E., Paton, J. J., &
174 Kampff, A. R. (2015). Bonsai: An event-based framework for processing and controlling data
175 streams. *Frontiers in Neuroinformatics*, 9, 7. <https://doi.org/10.3389/fninf.2015.00007>
- 176 Lopes, G., Farrell, K., Horrocks, E. A. B., Lee, C.-Y., Morimoto, M. M., Muzzu, T.,
177 Papanikolaou, A., Rodrigues, F. R., Wheatcroft, T., Zucca, S., Solomon, S. G., & Saleem,
178 A. B. (2021). Creating and controlling visual environments using BonVision. *eLife*, 10,
179 e65541. <https://doi.org/10.7554/eLife.65541>
- 180 Magland, J., Soules, J., Baker, C., & Dichter, B. (2024). Neurosift: DANDI exploration
181 and NWB visualization in the browser. *Journal of Open Source Software*, 9(97), 6590.
182 <https://doi.org/10.21105/joss.06590>
- 183 Mano, O., Choi, M., Tanaka, R., Creamer, M. S., Matos, N. C. B., Shomar, J. W., Badwan,
184 B. A., Clandinin, T. R., & Clark, D. A. (2023). Long-timescale anti-directional rotation in
185 *Drosophila* optomotor behavior. *eLife*, 12, e86076. <https://doi.org/10.7554/eLife.86076>
- 186 Mayorquin, H., Baker, C., Adkisson-Floro, P., Weigl, S., Trapani, A., Tauffer, L., Rübél,
187 O., & Dichter, B. (2025). NeuroConv: Streamlining neurophysiology data conversion
188 to the NWB standard. *Proceedings of the 24th Python in Science Conference*. <https://doi.org/10.25080/ceh4257>
- 189
- 190 Peirce, J., Gray, J. R., Simpson, S., MacAskill, M., Höchenberger, R., Sogo, H., Kastman,
191 E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior*
192 *Research Methods*, 51(1), 195–203. <https://doi.org/10.3758/s13428-018-01193-y>
- 193 Pelli, D. G. (1997). The VideoToolbox software for visual psychophysics: Transforming numbers
194 into movies. *Spatial Vision*, 10(4), 437–442. <https://doi.org/10.1163/156856897X00366>
- 195 Reiser, M. B., & Dickinson, M. H. (2008). A modular display system for insect behavioral
196 neuroscience. *Journal of Neuroscience Methods*, 167(2), 127–139. [https://doi.org/10.](https://doi.org/10.1016/j.jneumeth.2007.07.019)
197 [1016/j.jneumeth.2007.07.019](https://doi.org/10.1016/j.jneumeth.2007.07.019)
- 198 Rübél, O., Tritt, A., Ly, R., Dichter, B. K., Ghosh, S., Niu, L., Baker, P., Soltesz, I., Ng,
199 L., Svoboda, K., Frank, L., & Bouchard, K. E. (2022). The Neurodata Without Borders

- ecosystem for neurophysiological data science. *eLife*, 11, e78362. <https://doi.org/10.7554/eLife.78362>
- Shapcott, K. A., Weigand, M., Glukhova, M., Havenith, M. N., & Schölvinck, M. L. (2025). DomeVR: Immersive virtual reality for primates and rodents. *PLOS ONE*, 20(1), e0308848. <https://doi.org/10.1371/journal.pone.0308848>
- Štih, V., Petrucco, L., Kist, A. M., & Portugues, R. (2019). Stytra: An open-source, integrated system for stimulation, tracking and closed-loop behavioral experiments. *PLOS Computational Biology*, 15(4), e1006699. <https://doi.org/10.1371/journal.pcbi.1006699>
- Stowers, J. R., Hofbauer, M., Bastien, R., Griessner, J., Higgins, P., Farooqui, S., Fischer, R. M., Nowikovsky, K., Haubensak, W., Couzin, I. D., Tessmar-Raible, K., & Straw, A. D. (2017). Virtual reality for freely moving animals. *Nature Methods*, 14(10), 995–1002. <https://doi.org/10.1038/nmeth.4399>
- Straw, A. D. (2008). Vision Egg: An open-source library for realtime visual stimulus generation. *Frontiers in Neuroinformatics*, 2, 4. <https://doi.org/10.3389/neuro.11.004.2008>
- Teeters, J. L., Godfrey, K., Young, R., Dang, C., Friedsam, C., Wark, B., Asari, H., Peron, S., Li, N., Peyrache, A., Denisov, G., Siegle, J. H., Olsen, S. R., Martin, C., Chun, M., Tripathy, S., Blanche, T. J., Harris, K., Buzsáki, G., ... Sommer, F. T. (2015). Neurodata without borders: Creating a common data format for neurophysiology. *Neuron*, 88(4), 629–634. <https://doi.org/10.1016/j.neuron.2015.10.025>
- The MWorks Project. (2026). *MWorks: A framework for conducting realtime neurophysiology and psychophysics experiments*. <https://mworks.github.io/>
- Turner, M. H., Krieger, A., Pang, M. M., & Clandinin, T. R. (2022). Visual and motor signatures of locomotion dynamically shape a population code for feature detection in *Drosophila*. *eLife*, 11, e82587. <https://doi.org/10.7554/eLife.82587>
- Weddington, J. C., Au, D. D., Melander, J. B., Faragalla, Y., Alaoui, Z., & Baccus, S. A. (2026). Parallel construction of object motion from retina to cortex. *bioRxiv*. <https://doi.org/10.64898/2026.07.27.740872>